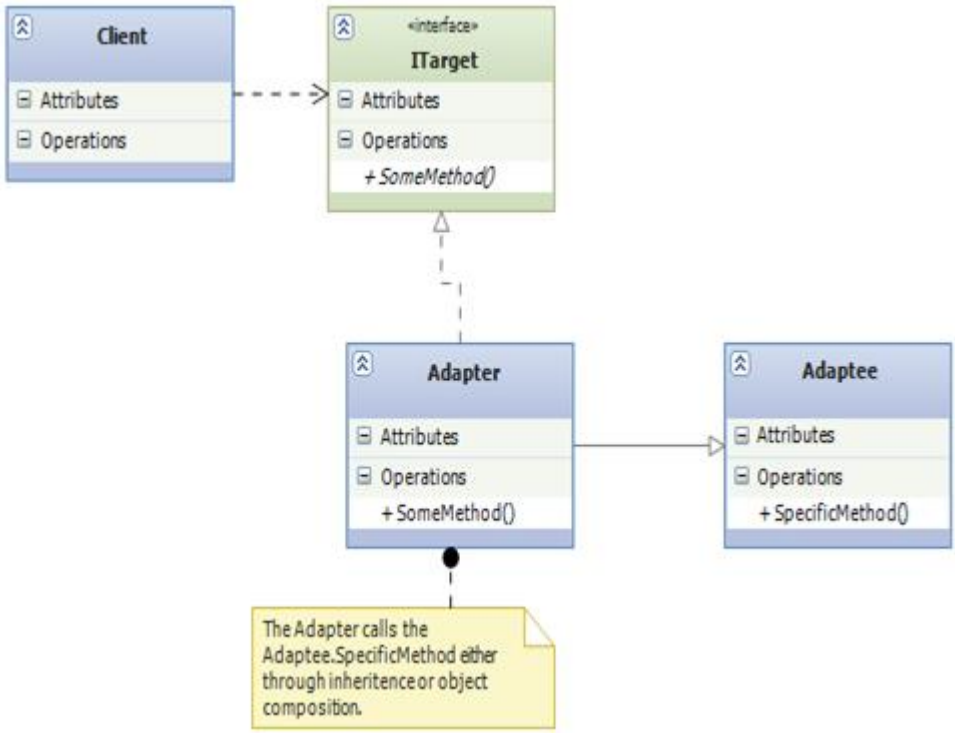
# Lab9: Adapter Pattern

**UML Diagram for Adapter Pattern.**




**Client:** This is the class which wants to achieve some functionality by using the adaptee's code.

**ITarget:** This is the interface which is used by the client to achieve functionality.

**Adapter:** This is the class which would implement ITarget and would call the Adaptee code which the client wants to call.

**Adaptee:** This is the functionality which the client desires but its interface is not compatible with the client.

## Lab Exercise

Start a new Visual Studio Project as console application (C#) which adds a new class1.cs, rename it a **Vendor.cs** (this is the client class in the pattern and you will add the code to take an input from user to enter the weight of the package and will access the Adaptee class (CanadaPost) which has the **PostalCost()** method to calculate the shipping costs.

Let us implement the adapter pattern for an online store which wants to calculate the shipping costs for the packages for customer orders. The shipping costs are however determined by the shipper; Canada Post in this case. The client (online store) cannot directly call the **PostalCost()** method of the adaptee (Canada Post) so we will implement the Adapter Pattern.

Following is the code for the other three elements of the pattern (2 classes and One Interface, add a new class/interface and copy the following code for all three of these one by one

**Adaptee: (Canada Post)**

I have created a very simple method to calculate the shipping costs based on package weight and have provided the code at the end of this document (you can copy this code in the adaptee class).

```
class CanadaPost
{public double PostalCost(double packageWeight)
    {
        double shipcost;
```

```
            if (packageWeight <= 5)
            { shipcost = 3.50; }

            else if (packageWeight <= 10)
            { shipcost = 5.00; }


            else if (packageWeight <= 15)
            { shipcost = 7.50; }


            else if (packageWeight <= 20)
            { shipcost = 10.00; }

            else
            { shipcost = 15.00; }

            return shipcost;
        }


    }
```

**I Target (Interface)**

```
//create as an Interface

interface ITarget
    {
        double CalculateShippingCost(double wt);
    }
```

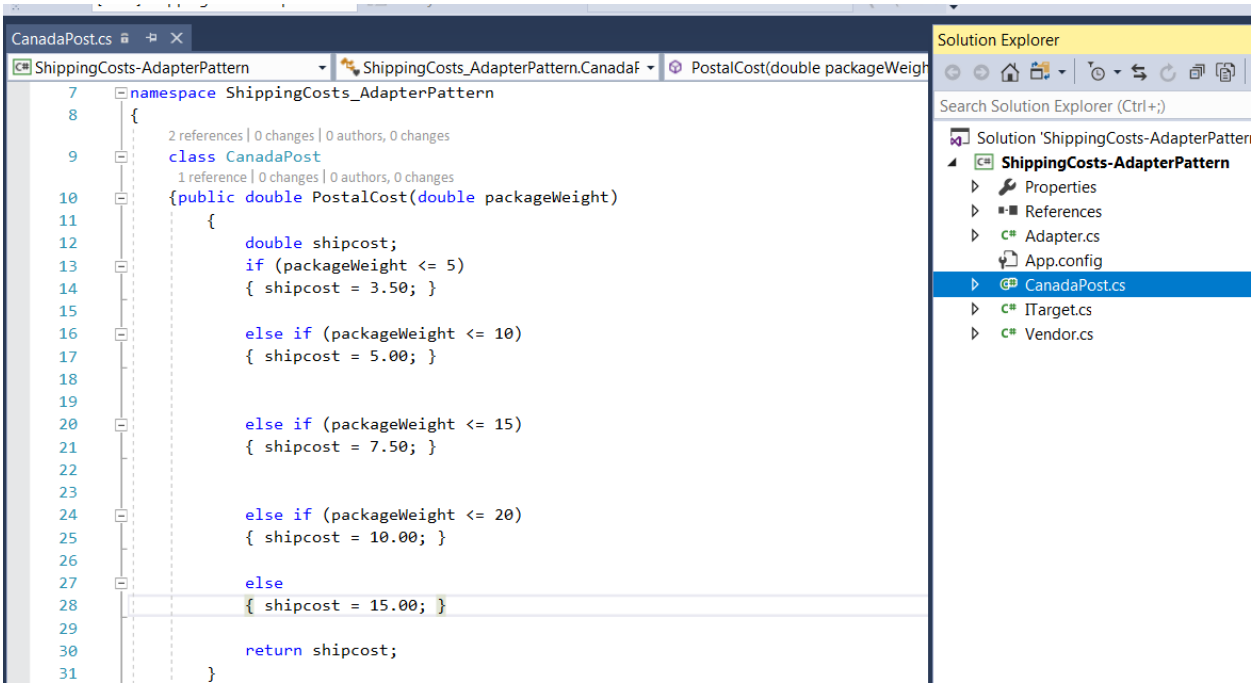
**Adapter**

```
class Adapter:ITarget

    {   public double CalculateShippingCost(double wt)
        {
            CanadaPost shipper = new CanadaPost();
            return shipper.PostalCost(wt);
        }
    }
```

**Screen shot for the Project**

Provided below is a screenshot for the adaptee classes (CanadaPost) and you will create the other two classes and one Interface (based on the following example).



**Client: (Vendor.cs)**

When the user will enter the weight of the package on online stores website (client), it will use the adapter pattern to call the PostalCost() method from Shipper's class(Canada Post) through The **ITarget** interface and its implementing class; **the Adapter**

```
Please enter the weight of the package in Pounds (lb)
23
Total Shipping cost is:
15$
```

**Submission Process:**

***** PEASE DO NOT SUBMIT THE C# PROJECT

Please copy the code for only the **Vendor.cs** class in MS Word (and convert it to pdf) and upload that single file through the Lab9-submissionLink.

**References:**


1-https://www.codeproject.com/Articles/774259/Adapter-Design-Pattern-Csharp

2- https://books.google.ca/books?id=pD2XMZLGUAYC&pg=PA

3- https://en.wikipedia.org/wiki/Adapter_pattern